CMSI 2130 Midterm

Classical Search

5 Components

1. Initial state
2. Goal test/state
3. Actions
4. Transitions
5. Costs

Uniform vs. Non-
constant vs. varied action costs

Graph vs. Tree
uses graveyard vs. doesn't use

Combinatorial Search
study of problems w/ large search spaces (e.g., w/ many solutions or few that are hard to find).

Uninformed Search
proceduralized by tree construction, node visitation, child generation order

Informed Search
proceduralized by domain-specific knowledge (heuristic)

Completeness
guaranteed to find a solution when 1+ exist

Optimality
guaranteed to find least-cost solution

	Time	Space	Goal Test	DS	Graveyard
BFS	$O(b^d)$	$O(b^d)$	gen	queue	Y
DFS	$O(b^m)$	O(b•m)	gen	stack	N
DLFS	$O(b^m)$	O(b•m)	gen	stack	N
IDDFS	$O(b^d)$	O(b•d)	gen	stack	N
Best-first	$O(b^d)$	$O(b^d)$	exp	heap	Y
A*	$O(b^d)$	$O(b^d)$	exp	heap	Y

Notes

- Depth searches have more overhead but use far less space
- A* expands the fewest nodes, given a good heuristic
- DFS risks infinite loop
- DLFS only complete + optimal when m = d
- Only best-first + A* are optimal for **non-uniform** cost problems
- Best-first uses f(n) = g(n), while A* uses f(n) = g(n) + h(n).

Heuristics

Admissibility
breaks when h(n) > true cost to goal

Consistency
breaks when h(p) > transition cost from p to n + h(n), where p is previous node and n is current

Why Admissibility and Consistency Matter
When admissibility is broken, the heuristic may overestimate the cost of the optimal path, making it look worse than it is, so it's never expanded. When consistency is broken, if two paths intersect at a node, the algorithm might expand the longer path first, adding the node to the graveyard and preventing the shorter, optimal path from being explored.

Ensemble Heuristics
The best heuristic gets as close as possible to the true cost without overestimating. If you have multiple admissible heuristics, you can use them together in an **ensemble**:

```
def ensemble(...):
  return max(h1(...), h2(...), h3(...))
```

Adversarial Search

Canonical Games
2 players, **turn-based** (alternating), **perfect information** (fully observable; no probability), **constant-sum** (win-lose or tie-tie)

Minimax Algorithm
Game tree:

- All possible moves from all players to terminal states
- Utility scores for each player at each terminal state
- Alternates max and min nodes by depth (root is max)
- Child of root with max value is next action agent takes

α-β Pruning:

- Root starts with [α, β] = [-∞, ∞]
- Children inherit starting [α, β] from parent's current values
- Parent's [α, β] are updated **immediately** after evaluating a child
- **Max** nodes: only update α: α = max(α, v)
- **Min** nodes: only update β: β = min(β, v)
- **Prune** (skip any uninvestigated children) when β ≤ α

Dynamic Programming

Two Properties:

- **Optimal Substructure:** optimal solution can be found *as a function of* optimal solutions to subproblems
- **Overlapping Subproblems:** solutions to same subproblems required to solve large subproblems *multiple times*

Two Approaches:

- **Bottom-Up Dynamic Programming (Tabulation):** solves smallest subproblems first, building up to large and larger subproblems
- **Top-Down Dynamic Programming (Memoization):** start with the large problem, recursively identifying specific subproblems that need to be solved

CMSI 2130 Midterm

Changemaker

Create an r by c table, where r is the number of different coins and c is the change to make. Each row introduces a new coin. Each cell represents the number of coins used to make the change. To fill in any (r,c), use the formula:

```
T[r][c] = {
    0 if c == 0, # base case
    T[r-1][c] if D[r] > c,
    min( # else
        T[r-1][c],
        1 + T[r][c - D[r]]
    )
}
```

Bottom-Up: start at the top left and fill in every cell.

Top-Down: start at the bottom right and only fill in necessary cells (recurse until all leaves are base case).

Tracing the Solution:

1. Start at the bottom-right cell.
2. Follow the arrows to see which previous cell produced optimal value.
3. Each time you trace a transition that uses a coin (when you go left), add that coin to the solution.
4. Continue until N = 0.

Longest Common Subsequence (LCS)

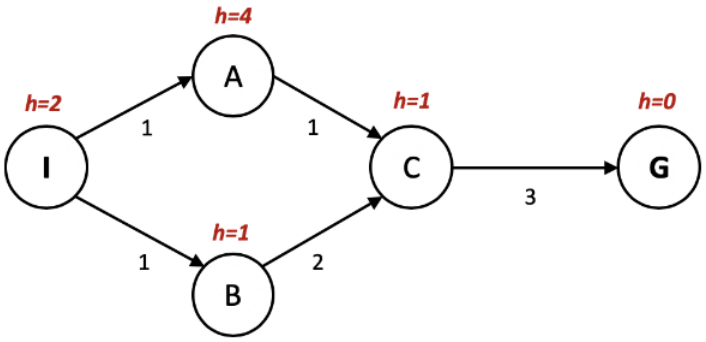
Create a table where one word is the rows and another word is the cols, with the first row/col empty space: ∅. Each cell represents the length of the LCS of the prefixes R[1..r] and C[1..c]. To fill in any (r,c), use the formula:

```
T[r][c] = {
    0 if r == 0 or c == 0, # base case
    # Mismatch
    max(T[r-1][c], T[r][c-1]) if R[r] != C[c],
    # Match
    T[r-1][c-1] + 1 if R[r] == C[c]
}
```

Tracing the solution: 1. Start at the bottom-right. **2.** If chars match, add char to solution and move diagonally up-left. **3.** If chars do not match, move either left or up, whichever has largest value. **4.** Repeat until r == 0 or c == 0. **5.** Reverse chars to get answer.

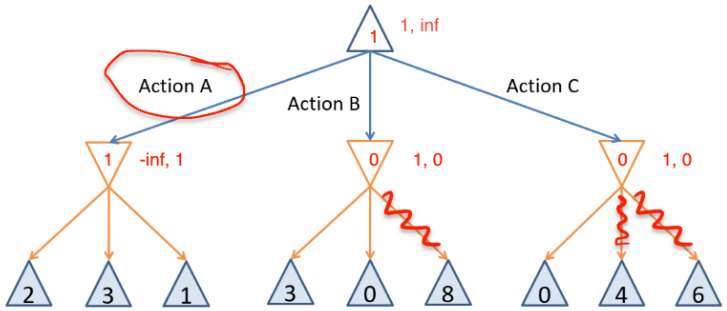
Diagrams

Heuristic Example Problem



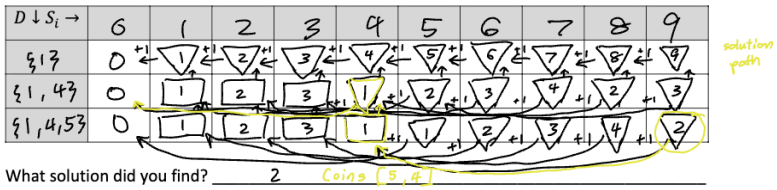
This heuristic **is admissible** because no nodes overestimate the cost to G. However, it is **not consistent**, because h(A) > cost(A -> C) + h(C).

Minimax



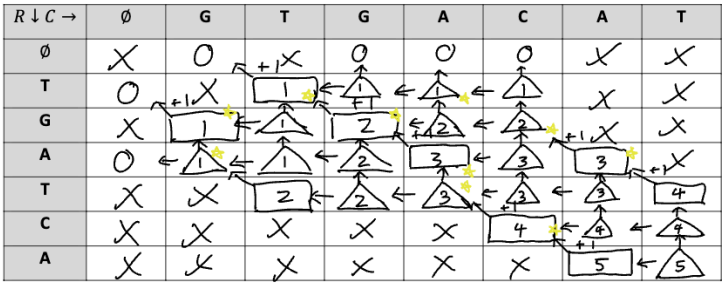
Note: comma-separated pairs represent each node's [α, β] after the algorithm is complete. Crossed-out paths are pruned. The circled action is chosen for the agent's next move.

Changemaker



Note: this is **bottom-up** changemaker. The solution is traced in yellow.

LCS



Note: this is **top-down** LCS. Unfilled cells are marked with "X". Cells that save effort (overlapping subproblems) are marked with "*".